

Fraud Detection Platform

Cahier des charges technique et feuille de route

Objectif : concevoir une plateforme de détection de transactions frauduleuses couvrant l'ensemble du cycle de vie d'un modèle ML, de la donnée jusqu'au déploiement et au monitoring.

Domaine	Machine Learning / MLOps / Backend / Cloud
Type	Projet End-to-End orienté production
Problème	Classification de transactions : fraude vs transaction légitime
Objectif principal	Produire une prédiction fiable, reproductible, testable, déployable et observable
Cible de latence	< 50 ms par requête d'inférence dans l'environnement cible, à valider par benchmark

1. Périmètre et objectifs

- Détecter automatiquement les transactions susceptibles d'être frauduleuses.
- Construire un pipeline de données reproductible : ingestion, validation, nettoyage, transformation et feature engineering.
- Comparer plusieurs modèles ML et sélectionner le meilleur compromis entre détection de fraude et fausses alertes.
- Tracer les expériences, paramètres, métriques et artefacts avec MLflow.
- Exposer le modèle via une API FastAPI documentée et validée.
- Conteneuriser l'ensemble avec Docker et automatiser les contrôles avec GitHub Actions.
- Préparer puis réaliser un déploiement cloud sur AWS, avec logs et monitoring.

Principe directeur : le projet doit ressembler à un petit produit ML professionnel, et non à un simple notebook d'entraînement.

2. Contexte métier et contraintes

On considère une fintech ou une banque en ligne traitant un volume important de transactions. Une transaction peut contenir des informations telles que le montant, le temps, la localisation, le moyen de paiement et des variables comportementales ou historiques.

Sortie attendue : une probabilité de fraude et une décision binaire selon un seuil configurable.

Contrainte importante : le problème est fortement déséquilibré. Sur le dataset de référence Credit Card Fraud Detection, la fraude représente environ 0,17 % des observations. L'Accuracy ne doit donc pas être la métrique principale.

Métriques prioritaires : PR-AUC et Recall, complétées par Precision, F1-score, ROC-AUC et matrice de confusion. Le seuil de décision devra être justifié selon le coût relatif d'une fraude manquée et d'une fausse alerte.

3. Données et pipeline

Dataset de départ : Credit Card Fraud Detection (Kaggle), utilisé comme dataset de référence pour la première version. Le projet devra documenter ses limites et ne pas présenter ce dataset comme représentatif de toutes les fraudes bancaires réelles.

- Ingestion : récupérer les données et définir une convention claire pour les sources et versions.
- Validation : contrôler schéma, types, valeurs manquantes, doublons, valeurs aberrantes et distribution de la cible.
- Préprocessing : nettoyage et transformations reproductibles ; appliquer le scaling uniquement lorsque le modèle l'exige.
- Feature engineering : construire les variables utiles sans fuite de données (data leakage).
- Split : séparer train/validation/test de façon reproductible et adaptée au contexte des données.
- Reproductibilité : toutes les étapes doivent être exécutables par script, sans dépendre d'opérations manuelles réalisées uniquement dans un notebook.
- Stockage : PostgreSQL pourra servir de couche de données applicative/transactionnelle ; les datasets bruts et artefacts volumineux seront plutôt conservés dans un stockage objet tel que S3 en environnement AWS.

4. Machine Learning

Approche : commencer par une baseline simple, puis comparer des modèles plus performants.

Étape	Modèle	Rôle
Baseline	Logistic Regression	Référence simple, interprétable et rapide
Comparaison	Random Forest	Modèle non linéaire de référence
Modèle avancé	XGBoost	Candidat principal pour la performance
Option	LightGBM	À intégrer si son bénéfice est démontré

- Traiter explicitement le déséquilibre des classes : pondération, stratégie de sampling ou méthode appropriée, avec justification expérimentale.
- Comparer les modèles à seuil comparable et analyser la courbe Precision-Recall.
- Choisir le modèle final sur des critères métier et techniques, pas uniquement sur un score unique.
- Conserver le pipeline de preprocessing avec le modèle afin d'éviter les divergences entre entraînement et inférence.

5. Expérimentation et MLflow

Chaque entraînement doit être traçable et reproductible.

- Parameters : modèle, learning rate, profondeur, nombre d'arbres, seed, stratégie de gestion du déséquilibre, seuil, etc.
- Metrics : PR-AUC, Recall, Precision, F1, ROC-AUC, matrice de confusion et temps d'inférence lorsque pertinent.
- Artifacts : modèle, courbes, rapports d'évaluation et éventuellement jeux de métriques.
- Model Registry : enregistrer et versionner le meilleur modèle retenu.
- Reproductibilité : associer chaque modèle à son code, ses paramètres et sa version de données.

6. Architecture logicielle

L'organisation du dépôt doit séparer la recherche exploratoire du code destiné à la production.

```
Arborescence cible :
fraud-detection-platform/
├── .github/workflows/ci-cd.yml
├── data/
├── docker/
├── notebooks/
├── src/
│   ├── config.py
│   ├── data/pipeline.py
│   ├── models/train.py
│   ├── models/evaluate.py
│   └── api/{main.py, schemas.py}
├── tests/
├── .gitignore
├── README.md
└── requirements.txt
```

- notebooks/ : EDA et prototypes uniquement ; aucune logique critique ne doit rester exclusivement dans un notebook.
- src/ : code modulaire, réutilisable et testable.
- tests/ : tests unitaires, intégration API et tests de comportement du modèle.
- docker/ : Dockerfile et configuration Docker Compose pour l'environnement local.

7. API d'inférence

Technologie : FastAPI + Pydantic.

Endpoint principal : **POST /predict**

```
Réponse conceptuelle :
{ "transaction_id": "12345", "fraud_probability": 0.91, "prediction": "fraud" }
```

- Valider strictement les données d'entrée.
- Charger le modèle au démarrage de l'application plutôt qu'à chaque requête.
- Retourner une réponse structurée et des erreurs HTTP explicites.
- Fournir la documentation OpenAPI générée par FastAPI.
- Ajouter des logs utiles sans exposer de données sensibles.
- Mesurer la latence d'inférence et vérifier la cible de < 50 ms par benchmark reproductible.

8. Base de données et stockage

PostgreSQL : utilisé notamment pour les données structurées et/ou les métadonnées applicatives. MLflow peut également utiliser PostgreSQL comme backend store.

AWS S3 : stockage des datasets, modèles et artefacts lorsque le projet passe en environnement cloud.

AWS RDS : alternative gérée à PostgreSQL pour la version cloud.

Principe : ne pas transformer PostgreSQL en stockage universel ; choisir le support selon le type, le volume et le cycle de vie de la donnée.

9. Docker et environnement local

- Conteneuriser l'API.
- Orchestrer localement les services nécessaires avec Docker Compose : API, MLflow et PostgreSQL.
- Utiliser des variables d'environnement et ne jamais intégrer de secrets dans le dépôt.
- Permettre à une personne externe de cloner le dépôt et de lancer l'environnement avec un nombre minimal de commandes.

10. Tests et qualité logicielle

Unit tests : preprocessing, feature engineering, validation et fonctions de prédiction.

Integration tests : API → modèle et, lorsque nécessaire, API → base de données.

Model tests : vérifier le format du modèle, la compatibilité des features et un niveau minimal de performance défini à l'avance.

Quality : linting/formatage avec Ruff (ou équivalent) et conventions de code cohérentes.

11. CI/CD

À chaque push ou pull request, GitHub Actions doit au minimum exécuter :

- Installation des dépendances.
- Linting / vérification qualité.
- Tests unitaires et tests d'intégration.
- Build de l'image Docker.
- Éventuellement, publication de l'image et déploiement après validation.

Objectif : empêcher qu'un changement de code cassant les tests ou la construction Docker puisse être intégré comme version valide.

12. Déploiement AWS

Service	Usage prévu
S3	Datasets, modèles et artefacts
ECR	Registry des images Docker
EC2	Hébergement de l'API pour une première version
RDS	PostgreSQL managé
IAM	Gestion des identités et permissions
CloudWatch	Logs, métriques et observabilité
SageMaker	Option à étudier si une plateforme ML managée apporte une réelle valeur

Déploiement progressif : commencer par l'environnement local, puis publier l'image sur ECR et déployer une première version simple sur AWS. Les choix de services doivent rester proportionnés au budget et à l'objectif pédagogique.

13. Monitoring et observabilité

- API : nombre de requêtes, latence, taux d'erreur et disponibilité.
- Infrastructure : CPU, mémoire et état du service.
- ML : distribution des features, évolution des prédictions, data drift et, lorsque les labels deviennent disponibles, évolution des performances.
- Alerting : définir progressivement des seuils sur les erreurs et la latence.

Le monitoring du drift doit être présenté comme une capacité progressive : sur un dataset académique statique, certaines dimensions de monitoring réel ne peuvent pas être validées de manière réaliste sans données temporelles ou flux simulé.

14. Architecture cible

```
USER / CLIENT
↓
FastAPI → Validation → ML Inference → Prediction
↓ ↓ ↓
PostgreSQL MLflow CloudWatch

AWS : S3 / ECR / EC2 / RDS / IAM / CloudWatch
```

Architecture finale : elle sera ajustée après expérimentation, en fonction des contraintes de coût, sécurité, latence, maintenabilité et scalabilité.

15. Feuille de route

Sprint	Livrables principaux
Sprint 1 — Cadrage & initialisation	Définir le problème et les métriques ; choisir/documenter le dataset ; créer le dépôt ; configurer Python, Git et la structure du projet.
Sprint 2 — Data Engineering	Ingestion ; EDA ; validation ; nettoyage ; split ; preprocessing ; feature engineering ; première intégration PostgreSQL.
Sprint 3 — Machine Learning	Baseline ; Random Forest ; XGBoost ; éventuellement LightGBM ; gestion du déséquilibre ; comparaison des métriques ; sélection du modèle.
Sprint 4 — MLOps	MLflow ; tracking ; artefacts ; versioning ; Model Registry ; pipeline d'entraînement reproductible.
Sprint 5 — API	FastAPI ; schémas Pydantic ; chargement du modèle ; endpoint /predict ; gestion des erreurs ; documentation ; benchmark de latence.
Sprint 6 — Docker	Dockerfile ; Docker Compose ; configuration locale reproductible ; vérification de l'exécution de bout en bout.
Sprint 7 — CI/CD	GitHub Actions ; linting ; tests ; build Docker ; contrôles automatiques sur push/PR.
Sprint 8 — AWS	S3 ; ECR ; EC2/RDS ; IAM ; déploiement ; configuration des secrets et permissions.
Sprint 9 — Monitoring	CloudWatch ; logs ; métriques API ; latence ; erreurs ; premiers indicateurs de drift.
Sprint 10 — Finalisation	Optimisation ; sécurité ; coûts ; documentation ; architecture finale ; démonstration ; préparation du portfolio/CV.

16. Definition of Done

- Problème métier et limites du dataset documentés.
- Pipeline de données reproductible et exécutable hors notebook.
- Baseline + plusieurs modèles comparés avec métriques adaptées au déséquilibre.
- Modèle final et preprocessing versionnés dans MLflow.
- API FastAPI fonctionnelle, documentée et testée.
- Objectif de latence mesuré par benchmark.
- Application conteneurisée et environnement local reproductible.

- Tests unitaires et d'intégration automatisés.
- CI GitHub Actions fonctionnelle.
- Déploiement AWS opérationnel pour une première version.
- Logs et monitoring minimum disponibles.
- README permettant à un tiers de reproduire le projet.
- Architecture, choix techniques, coûts et limites documentés.
- Démonstration de bout en bout disponible.

17. Technologies

Core : Python, SQL, Linux, Git

Data : Pandas, NumPy, PostgreSQL

ML : Scikit-learn, XGBoost, éventuellement LightGBM

MLOps : MLflow, Model Registry

Backend : FastAPI, Pydantic

Engineering : Docker, Docker Compose, Pytest, Ruff, GitHub Actions

Cloud : AWS S3, ECR, EC2, RDS, IAM, CloudWatch ; SageMaker à évaluer

18. Utilisation de l'IA

ChatGPT, Claude, Gemini, Cursor ou GitHub Copilot peuvent être utilisés comme assistants d'ingénierie et d'apprentissage.

- Faire expliquer une technologie ou une fonction avant de l'implémenter.
- Faire challenger l'architecture : sécurité, coût, scalabilité, maintenabilité.
- Utiliser l'IA pour le debugging, puis comprendre la cause avant d'appliquer une correction.
- Demander des code reviews et des tests supplémentaires.
- Simuler des entretiens techniques et défendre soi-même les choix réalisés.

Règle : aucune partie critique du projet ne doit être intégrée sans compréhension, vérification et capacité à l'expliquer.

19. Livrables finaux et valeur CV

- Repository GitHub propre avec code, tests, documentation et historique cohérent.
- Pipeline ML reproductible avec suivi d'expériences et modèle versionné.
- API de prédiction dockerisée et documentée.
- Déploiement AWS avec stockage, registry, infrastructure et observabilité.
- Démonstration end-to-end montrant une transaction → validation → prédiction → logs.

Compétences démontrées : Python, SQL, Machine Learning, XGBoost, MLOps, MLflow, FastAPI, Docker, CI/CD, PostgreSQL, AWS, monitoring et software engineering.

Positionnement portfolio : présenter le projet comme une expérience d'End-to-End ML Engineering / MLOps plutôt que comme un simple projet de classification.

Document de cadrage — version initiale. Les choix d'implémentation pourront être ajustés après les premières expérimentations, benchmarks et contraintes de coût.